When Does HTTP/3 Improve Tail Latency Over HTTP/2 Under Network Impairments

Song Ni,

***Abstract*—HTTP/3, built on QUIC, is designed to improve web performance by eliminating head-of-line blocking and improving loss recovery. However, its benefits are known to depend on network conditions and workload characteristics. In this project, we conduct controlled experiments to evaluate when HTTP/3 outperforms HTTP/2 under varying latency and packet loss conditions. Using a local testbed with Linux tc netem, we emulate different RTT and loss environments and measure completion time and p95 latency for a many-small-object workload. Our results show that HTTP/3 does not provide significant benefits under pure latency conditions and may incur slight overhead. However, under increasing packet loss, HTTP/3 demonstrates improved robustness and more stable tail latency compared to HTTP/2. These findings highlight that the advantages of HTTP/3 are workload- and condition-dependent**

***Index Terms*—HTTP/2, HTTP/3, QUIC, tail latency, packet loss, RTT**

I. Introduction

MODERN web performance is strongly influenced by transport-layer protocols and their behavior under network impairments. HTTP/2 improves efficiency through multiplexing over TCP, but suffers from head-of-line (HOL) blocking, where a single packet loss can stall all in-flight streams. HTTP/3, built on QUIC, addresses this limitation by providing stream-level independence and improved loss recovery mechanisms [1], [2].

Prior studies suggest that QUIC can mitigate the impact of TCP-level HOL blocking and may exhibit more stable performance in lossy or highly variable network environments [3], [4]. However, these benefits are not universal and depend on factors such as congestion control, network conditions, and workload characteristics. In particular, most evaluations focus on average performance, while recent work highlights that tail latency (e.g., p95/p99) is often more critical for user-perceived performance in interactive workloads [5], [6].

Motivated by these observations, this work investigates the following research question: Under what network conditions does HTTP/3 reduce completion time and tail latency compared to HTTP/2 for high-concurrency small-object workloads? We hypothesize that HTTP/3 provides limited benefits under pure latency conditions, but improves tail latency under packet loss due to the absence of HOL blocking.

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see http://www.michaelshell.org/contact.html).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.

II. Method

We conduct controlled experiments using a local testbed to evaluate HTTP/2 and HTTP/3 performance under varying network conditions. The server is implemented using Caddy, which supports both HTTP/2 and HTTP/3, and serves a fixed set of static objects. The client-side implementation uses HTTPX with HTTP/2 enabled and aioquic for HTTP/3, ensuring that both protocols are evaluated within comparable environments.

To ensure fair comparison, each experimental run establishes a single connection (or session) and downloads all objects within that connection. This design allows HTTP/2 multiplexing and HTTP/3 stream-level concurrency to operate as intended.

We evaluate two workload types. The primary workload consists of many small objects (300 files), which is representative of modern web page loading behavior. We also include a secondary few-large-object workload (10 objects of approximately 1MB each) to study workload sensitivity.

Network conditions are controlled using Linux `tc netem`. We vary round-trip time (RTT) across {20, 100, 200} ms and packet loss across {0%, 1%, 3%, 5%}. Each configuration is repeated 20 times to reduce randomness.

We measure total completion time for each workload and compute both median and p95 latency. Median values capture typical performance, while p95 highlights tail behavior under adverse conditions.

III. Result

A. Many-small workload

We first analyze performance within each workload, and then compare behavior across workloads. Figure 1 illustrates the performance of HTTP/2 and HTTP/3 under varying network conditions for the many-small workload.

1) Completion time vs RTT: Under the many-small workload, both HTTP/2 and HTTP/3 exhibit increasing completion time as RTT increases. HTTP/3 is consistently slightly slower than HTTP/2 across all RTT values. This indicates that under latency-only conditions, HTTP/3 does not provide significant benefits. The additional QUIC handshake and implementation overhead may contribute to this slight performance penalty.

2) p95 completion time vs loss: Under increasing packet loss, HTTP/2 exhibits more significant degradation in tail latency, while HTTP/3 shows a more gradual increase and more stable behavior across loss levels. Although HTTP/3 does not consistently achieve lower completion time than HTTP/2, it demonstrates improved robustness by avoiding

sharp performance spikes. This behavior is consistent with the expected benefit of QUIC in mitigating head-of-line blocking.

B. Few-large workload

Figure 2 shows the performance under the few-large workload.

1) Completion time vs RTT: In the few-large workload, both HTTP/2 and HTTP/3 show similar scaling behavior as RTT increases. The performance gap between the two protocols is relatively small, indicating that RTT primarily affects bulk data transfer rather than protocol-level differences.

2) p95 completion time vs loss: Under packet loss, both protocols exhibit gradual performance degradation, but HTTP/3 does not show a clear advantage over HTTP/2. This suggests that for large-object transfers, performance is dominated by data transfer efficiency and implementation overhead rather than stream-level independence.

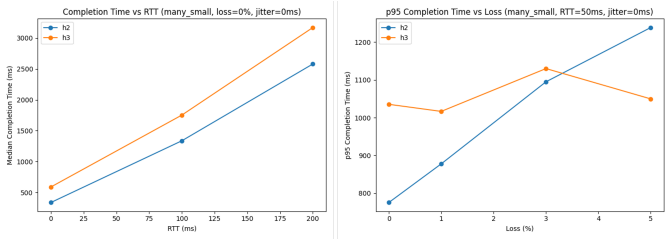

Fig. 1. Performance of HTTP/2 and HTTP/3 under varying network conditions for the many-small workload. The left panel shows median completion time vs RTT, and the right panel shows p95 completion time vs packet loss.

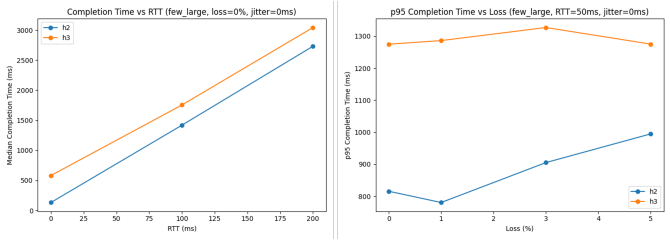

Fig. 2. Performance of HTTP/2 and HTTP/3 under varying network conditions for the few-large workload. The left panel shows median completion time vs RTT, and the right panel shows p95 completion time vs packet loss.

IV. Discussion

Our results show that HTTP/3 provides benefits primarily under lossy conditions, where its stream-level independence prevents a single lost packet from blocking all streams. This leads to improved tail latency stability under packet loss. However, HTTP/3 does not consistently achieve lower completion time than HTTP/2 in our experiments.

Under pure RTT conditions with no packet loss, HTTP/3 does not outperform HTTP/2. This is expected because head-of-line blocking is not triggered, and QUIC introduces additional handshake and processing overhead.

In the few-large-object workload, we observe that performance is dominated by client implementation overhead rather than protocol-level differences. As a result, HTTP/3 does not show clear advantages in this setting. This further confirms that HTTP/3 benefits are workload-dependent and most pronounced in high-concurrency small-object scenarios.

Finally, we observe non-monotonic behavior in p95 latency under packet loss. This is due to the stochastic nature of packet loss and the sensitivity of tail metrics to outliers. Even with repeated runs, small variations in packet loss patterns can significantly affect tail latency measurements.

V. Conclusion

In this study, we evaluated HTTP/2 and HTTP/3 under controlled network conditions. Our results show that HTTP/3 does not significantly outperform HTTP/2 under pure latency. However, under packet loss, HTTP/3 provides improved tail latency and greater robustness. These findings demonstrate that HTTP/3 advantages are condition- and workload-dependent rather than universal.

Appendix A
Proof of the First Zonklar Equation

Appendix one text goes here.

References

[1] Jana Iyengar, Martin Thomson, et al. Quic: A udp-based multiplexed and secure transport. Technical report, RFC 9000, may, 2021.

[2] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, et al. The quic transport protocol: Design and internet-scale deployment. In *Proceedings of the conference of the ACM special interest group on data communication*, pages 183–196, 2017.

[3] Arash Molavi Kakhki, Samuel Jero, David Choffnes, Cristina Nita-Rotaru, and Alan Mislove. Taking a long look at quic: an approach for rigorous evaluation of rapidly evolving transport protocols. In *proceedings of the 2017 internet measurement conference*, pages 290–303, 2017.

[4] Jashanjot Singh Sidhu, Jorge Ignacio Sandoval, Abdelhak Bentaleb, and Sandra Céspedes. From 5g ran queue dynamics to playback: A performance analysis for quic video streaming. *IEEE Transactions on Networking*, 34:2384–2399, 2025.

[5] Jeffrey Dean and Luiz André Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[6] Jialin Li, Naveen Kr Sharma, Dan RK Ports, and Steven D Gribble. Tales of the tail: Hardware, os, and application-level sources of tail latency. In *Proceedings of the ACM Symposium on Cloud Computing*, pages 1–14, 2014.